APPENDIX A
FORMAL ASSERTION CATALOG

Table X catalogs the 277 non-commented source-level `assert property` statements according to the eight property families defined in Table III. Assertions inside generate loops are listed once by source label, and parameter-conditional assertions are included even when a particular configuration does not elaborate them. The final integrated configuration expands the applicable templates across managers and protected subordinates and elaborates 409 assertion instances.

Table X
FORMAL ASSERTION CATALOG

Family 1: AXI4 Protocol and Framing

Assertion	Description
ast_int_s_aw_capture_first	Requires an integrated write address to be captured without stalling.
ast_int_s_w_stable	Keeps write-data payload stable while the integrated input is backpressured.
ast_int_s_ar_capture_first	Requires an integrated read address to be captured without stalling.
ast_int_wlast_after_aw	Checks WLAST against the expected write-beat count after AW acceptance.
ast_int_wlast_with_aw	Checks WLAST framing when AW and W are accepted together.
ast_int_w_before_aw_framing	Checks that W-before-AW beats are consistent with the later AWLEN.
ast_int_m_b_causal	Allows a downstream B response only after the corresponding allowed write completes, with the correct BID.
ast_m_aw_stable	Keeps forwarded AWVALID and its payload stable under backpressure.
ast_m_w_stable	Keeps forwarded WVALID, WDATA, WSTRB, and WLAST stable under backpressure.
ast_s_b_stable	Keeps the returned B response and BID stable under manager backpressure.
ast_m_ar_stable	Keeps forwarded ARVALID and its payload stable under backpressure.
ast_s_r_stable	Keeps the returned R payload stable under manager backpressure.
ast_m_b_immediate_capture	Requires every legal downstream BVALID to be accepted immediately.
ast_manager_b_stable	Keeps each manager-facing B response stable while that manager stalls.
ast_manager_r_stable	Keeps each manager-facing R response stable while that manager stalls.
ast_subordinate_aw_stable	Keeps each protected-subordinate AW request stable under backpressure.
ast_subordinate_w_stable	Keeps each protected-subordinate W transfer stable under backpressure.
ast_subordinate_ar_stable	Keeps each protected-subordinate AR request stable under backpressure.
ast_captured_aw_not_represented	Prevents a captured write address from being presented to the firewall again.
ast_captured_ar_not_represented	Prevents a captured read address from being presented to the firewall again.
ast_wlast_closes_manager_w_channel	Stops presenting write data after the accepted WLAST beat.

Family 2: Arbitration and Ownership

Assertion	Description
ast_idle_silence	Requires no grant when no arbitration request is active.
ast_mutual_exclusion	Allows at most one arbiter grant at a time.
ast_work_conservation	Requires a grant whenever at least one request is active.
ast_grant_validity	Allows a grant only to a requester that is currently requesting.
ast_prio_highest_exact	Requires the highest-priority requester to receive the grant whenever it requests.
ast_prio_strict_adherence	Requires each lower-priority grant to yield to every active higher-priority request.
ast_owner_mutex	Prevents simultaneous write and read transaction ownership.
ast_no_aw_ar_same_cycle	Prevents simultaneous acceptance of write and read addresses.
ast_no_w_ar_same_cycle	Prevents simultaneous acceptance of write data and a read address.
ast_write_blocks_read	Blocks read-channel activity while a write transaction owns the firewall.
ast_read_blocks_write	Blocks write-channel activity while a read transaction owns the firewall.
ast_no_second_aw_accept	Prevents acceptance of a second write address while one is active.
ast_no_second_ar_accept	Prevents acceptance of a second read address while one is active.
ast_write_priority	In standalone firewall mode, gives a pending write precedence over a simultaneous read when the firewall is idle.
ast_arbiter_request_boundary	Forms arbitration requests only when no owner or cancellation fence is active.
ast_fixed_priority_exact	Requires the integrated arbiter grant to match the fixed-priority reference model.
ast_onehot_manager_admission	Allows at most one manager to receive transaction acceptance in a cycle.
ast_onehot_manager_response	Allows at most one manager to receive a B response and at most one to receive an R response.
ast_no_read_write_accept_same_cycle	Prevents simultaneous read and write admission across all managers.
ast_aw_w_same_source	Requires simultaneous AW and W handshakes to come from the same manager.

Table X (continued)

Assertion	Description
ast_new_grant_is_accepted	Requires every issued grant to produce an accepted firewall-side transaction transfer.
ast_selected_manager_write_priority	Gives the selected manager's pending write precedence over its simultaneous read.
ast_fw_handshakes_match_manager_ports	Requires firewall-side handshakes to match the aggregate handshakes at the manager ports.
ast_ready_only_to_selected_manager	Drives request readiness only to the currently selected manager.
ast_locked_owner_excludes_other_requests	Prevents nonowners from being accepted while transaction ownership is locked.
ast_bvalid_only_to_write_owner	Returns each B response only to the locked write owner with matching payload.
ast_rvalid_only_to_read_owner	Returns each R response only to the locked read owner with matching payload.
ast_owner_bready_is_selected	Uses only the write owner's BREADY for the firewall response.
ast_owner_rready_is_selected	Uses only the read owner's RREADY for the firewall response.
ast_nonowner_bvalid_low	Keeps BVALID low for every nonowner manager.
ast_nonowner_rvalid_low	Keeps RVALID low for every nonowner manager.
ast_owner_write_persists_without_b	Keeps write ownership unchanged until the write response completes.
ast_owner_read_persists_without_final_r	Keeps read ownership unchanged until the final read beat completes.
ast_w_before_aw_locks_source	Keeps a W-before-AW transaction associated with its original manager.
Family 3: Trusted Identity and Context	
Assertion	**Description**
ast_user_width	Checks that AXI USER metadata width exactly accommodates MANAGER_ID and SUBORDINATE_ID.
ast_policy_req_has_owner	Allows a policy request only while one unresolved transaction owns the policy interface.
ast_policy_req_payload	Requires the policy request fields to match the captured transaction context.
ast_configuration_widths	Checks valid system dimensions and identifier, index, USER-field, and timeout parameter widths.
ast_fw_aw_source_and_trusted_user	Forwards AW from the selected manager while replacing its identity with the trusted physical-port MANAGER_ID.
ast_fw_w_source	Requires forwarded write data to come from the selected manager without payload alteration.
ast_fw_ar_source_and_trusted_user	Forwards AR from the selected manager while replacing its identity with the trusted physical-port MANAGER_ID.
ast_policy_request_has_captured_owner	Requires every policy request to match the captured owner and transaction context.
ast_policy_manager_is_physical_owner	Requires the policy MANAGER_ID to equal the trusted identity of the physical transaction owner.
ast_policy_subordinate_is_selected_manager_metadata	Requires the policy SUBORDINATE_ID to equal the subordinate metadata captured from the selected manager.
Family 4: Burst Legality and Address Range	
Assertion	**Description**
ast_live_write_footprint	Requires an accepted write address to produce the corresponding write policy address range unless it times out.
ast_live_read_footprint	Requires an accepted read address to produce the corresponding read policy address range unless it times out.
ast_captured_write_footprint	Requires a write policy request to match the captured write address, IDs, and computed address range.
ast_captured_read_footprint	Requires a read policy request to match the captured read address, IDs, and computed address range.
ast_direct_fixed_footprint	Checks the computed address range for a representative FIXED burst.
ast_direct_unaligned_incr_footprint	Checks the computed address range for a representative unaligned INCR burst.
ast_direct_wrap_footprint	Checks the computed address range for a representative WRAP burst.
ast_direct_4k_crossing_denied	Marks a representative 4-KiB-crossing burst as illegal in the policy range.
ast_direct_reserved_burst_denied	Marks a reserved AXI burst type as illegal in the policy range.
ast_direct_fixed_length_denied	Marks an unsupported FIXED-burst length as illegal in the policy range.
ast_direct_wrap_length_denied	Marks an invalid WRAP-burst length as illegal in the policy range.
ast_direct_wrap_alignment_denied	Marks a misaligned WRAP burst as illegal in the policy range.
ast_direct_illegal_read_denied	Marks a representative illegal read burst as illegal in the policy range.
ast_direct_oversize_denied	Marks a transfer size wider than the data bus as illegal in the policy range.

Table X (continued)

Assertion	Description
ast_direct_address_overflow_denied	Marks a burst whose computed address range overflows the address space as illegal.
ast_write_policy_footprint	Requires the integrated write policy range to match the captured write burst.
ast_read_policy_footprint	Requires the integrated read policy range to match the captured read burst.
Family 5: Authorization and Policy Handling	
Assertion	**Description**
ast_policy_req_stable	Keeps the policy request and all request fields stable while backpressured.
ast_idle_policy_neutral	Drives neutral policy-request fields whenever POLICY_REQ_VALID is low.
ast_policy_req_is_presented	Presents a policy request for every active unresolved transaction that has not timed out.
ast_one_policy_req	Allows at most one accepted policy request per transaction.
ast_policy_req_drops_after_handshake	Deasserts the policy request after its handshake.
ast_waiting_aw_live_footprint	Prevents policy evaluation before the write address is captured.
ast_no_policy_req_from_w_alone	Prevents write data alone from creating a policy request.
ast_policy_rsp_ready_only_when_expected	Asserts policy-response readiness only for an outstanding accepted request.
ast_no_policy_rsp_before_req	Prevents acceptance of a policy response before its request has been accepted.
ast_one_policy_rsp	Allows at most one accepted policy response per transaction.
ast_policy_rsp_registered_ready	Registers policy-response readiness after request acceptance unless the transaction times out.
ast_policy_rsp_ready_sticky	Keeps policy-response readiness asserted while waiting for a response.
ast_policy_rsp_ready_drops	Deasserts policy-response readiness after a response handshake.
ast_policy_count_bounds	Bounds policy request, response, timeout, cancellation, and resolution counts and enforces their consistency.
ast_policy_completion_balanced	Requires policy accounting to be balanced when the AXI transaction completes.
ast_retired_policy_history_balanced	Requires every retired policy generation to have a consistent request, response, timeout, and cancellation history.
ast_policy_allow_ignored_without_handshake	Prevents an unhandshaken policy Allow value from enabling downstream or local completion activity.
ast_write_blocked_before_policy_resolution	Blocks write forwarding and completion before a write authorization decision exists.
ast_read_blocked_before_policy_resolution	Blocks read forwarding and completion before a read authorization decision exists.
ast_illegal_write_allow_isolated	Overrides an external Allow for an illegal write and keeps the protected side isolated.
ast_illegal_read_allow_isolated	Overrides an external Allow for an illegal read and keeps the protected side isolated.
ast_no_policy_request_from_w_alone	Prevents the integrated path from issuing a policy request for W data without a captured AW.
Family 6: Timeout, Cancellation, and Revocation	
Assertion	**Description**
ast_policy_cancel_stable	Keeps POLICY_CANCEL_VALID asserted while cancellation is backpressured.
ast_policy_timeout_configuration	Requires a positive timeout length whenever policy timeout is enabled.
ast_one_cycle_write_times_out_on_accept	For a one-cycle timeout, times out a write immediately after address acceptance without issuing a policy request.
ast_one_cycle_read_times_out_on_accept	For a one-cycle timeout, times out a read immediately after address acceptance without issuing a policy request.
ast_write_starts_policy_timer	Starts the policy timeout timer when a write address is accepted.
ast_read_starts_policy_timer	Starts the policy timeout timer when a read address is accepted.
ast_two_cycle_deadline_is_next_edge	For a two-cycle timeout, places the timeout event on the edge after address acceptance.
ast_disabled_timeout_never_runs	Keeps timeout and cancellation state inactive when policy timeout is disabled.
ast_w_before_aw_does_not_start_policy_timer	Prevents W-before-AW data from starting the policy timer before address acceptance.
ast_policy_timeout_counter_range	Keeps the active timeout counter within its configured legal range.
ast_policy_timeout_counter_advances	Advances the timeout counter by one cycle while awaiting policy resolution.
ast_timely_policy_response_stops_timeout	Stops the timer and records a response when policy resolution arrives before timeout.
ast_final_cycle_response_wins	Gives a valid policy response precedence when it handshakes on the final permitted cycle.

Table X (continued)

Assertion	Description
ast_final_cycle_without_response_times_out	Generates exactly one timeout resolution when the final permitted cycle ends without a response.
ast_policy_resolution_is_exclusive	Prevents a transaction from being resolved both by response and by timeout.
ast_policy_timeout_cannot_repeat	Prevents more than one timeout event or resolution for the same transaction.
ast_policy_timeout_sticky_until_safe_clear	Keeps timeout state asserted until local completion and any required cancellation make clearing safe.
ast_completed_quarantine_remains_sticky	Keeps an accepted timed-out transaction quarantined after AXI completion until cancellation retires it.
ast_timeout_before_request_needs_no_cancel	Requires no cancellation when timeout occurs before the policy request is accepted.
ast_timeout_after_request_enters_quarantine	Requires cancellation and response quarantine when timeout follows policy-request acceptance.
ast_policy_cancel_only_for_quarantine	Allows cancellation only for one accepted, unanswered policy request that has timed out.
ast_policy_cancel_excludes_response_ready	Prevents policy-response readiness while cancellation is active.
ast_policy_cancel_excludes_response_handshake	Prevents a policy-response handshake on the cancellation-retirement cycle.
ast_one_policy_cancel	Allows at most one cancellation handshake for a timed-out accepted request.
ast_policy_cancel_drops_after_handshake	Deasserts cancellation after the cancellation handshake.
ast_policy_generation_fence_sticky	Keeps a retired policy-generation fence active until a new request is accepted.
ast_quarantine_rejects_late_response	Rejects late policy responses while an accepted timed-out request remains quarantined.
ast_quarantine_blocks_policy_reuse	Blocks new policy requests and new address admission until quarantine retirement.
ast_completed_quarantine_blocks_new_w_owner	Blocks new write-data ownership after local completion while quarantine remains active.
ast_policy_response_never_accepted_after_timeout	Prevents acceptance of any policy response after timeout has been recorded.
ast_pre_timeout_revocation_is_not_late_revocation	Ignores revocation presented before an authorization decision as a late-revocation event.
ast_write_timeout_causes_initial_denial	Converts a timed-out write into an initial denial and blocks protected-side write activity.
ast_read_timeout_causes_initial_denial	Converts a timed-out read into an initial denial and blocks protected-side read activity.
ast_timeout_downstream_isolated	Prevents all protected-side AXI activity after policy timeout.
ast_write_timeout_discards_buffered_w	Discards a buffered write beat when the corresponding write times out.
ast_write_timeout_discards_boundary_w	Discards a write beat accepted on the timeout boundary.
ast_timed_out_write_drains_to_wlast	Continues accepting owner write data after timeout until WLAST is drained.
ast_write_timeout_existing_wlast_completion	Returns DECERR promptly when timeout occurs after WLAST is already available.
ast_write_timeout_final_w_completion	Returns DECERR promptly after the final drained write beat of a timed-out write.
ast_write_timeout_local_b	Requires the local response for a timed-out write to be DECERR with the original BID and no downstream write activity.
ast_read_timeout_first_local_beat	Generates a zero-data DECERR read beat after a read timeout.
ast_read_timeout_local_r	Requires every local read-timeout beat to carry the original RID, zero data, DECERR, and correct RLAST.
ast_read_timeout_final_conservation	Requires a timed-out read to complete with exactly the expected local beat count and no downstream read.
ast_simultaneous_write_allow_revoke_denied	Treats Allow coincident with revocation as an initial write denial.
ast_simultaneous_read_allow_revoke_denied	Treats Allow coincident with revocation as an initial read denial.
ast_simultaneous_allow_revoke_downstream_isolated	Prevents protected-side activity after an Allow decision that is vetoed by coincident revocation.
ast_simultaneous_write_revoke_discards_buffered_w	Discards buffered write data when coincident revocation vetoes initial write authorization.
ast_simultaneous_write_revoke_discards_response_edge_w	Discards write data accepted on the coincident-revocation decision edge.
ast_simultaneous_write_revoke_drains_to_wlast	Drains the denied write to WLAST after coincident revocation.

Table X (continued)

Assertion	Description
ast_simultaneous_write_revoke_existing_wlast_completion	Returns DECERR promptly when coincident revocation occurs with WLAST already available.
ast_simultaneous_write_revoke_final_w_completion	Returns DECERR promptly after the final drained beat of a write denied by coincident revocation.
ast_simultaneous_write_revoke_local_b	Requires the local response for a coincidently revoked write to be DECERR with no downstream write activity.
ast_simultaneous_read_revoke_first_local_beat	Generates a zero-data DECERR read beat when coincident revocation vetoes initial read authorization.
ast_simultaneous_read_revoke_local_r	Requires each locally generated read beat after coincident revocation to carry zero data, DECERR, and correct framing.
ast_simultaneous_read_revoke_final_conservation	Requires a coincidently revoked read to complete with exactly the expected local beat count and no downstream read.
ast_response_or_timeout_exclusive	Prevents the integrated policy transaction from being both responded to and timed out.
ast_timeout_matches_reference_deadline	Requires the integrated timeout event to produce the reference-model timed-out state.
ast_final_cycle_response_wins_integration	Requires a policy response on the deadline cycle to win over timeout in the integrated design.
ast_timeout_never_changes_owner	Keeps the transaction owner and operation type unchanged when timeout occurs.
ast_quarantine_blocks_all_new_admission	Blocks arbitration grants and new address admission while the cancellation fence is active.
ast_completed_quarantine_blocks_w_owner	Blocks new write-data admission after local completion while cancellation remains pending.
ast_accepted_timeout_completion_keeps_cancel_fence	Keeps the cancellation fence active after local completion until an accepted timed-out request is retired.
ast_quarantine_rejects_stale_response	Keeps policy-response readiness low while the integrated cancellation fence is active.
ast_cancel_retirement_precedes_reuse	Prevents policy-interface reuse until cancellation retires the timed-out accepted request.
Family 7: Write/Read Enforcement and Conservation	
Assertion	**Description**
ast_m_aw_only_if_allowed	Allows a downstream AW only for a captured, legal, policy-authorized write that has not already been forwarded.
ast_m_aw_payload	Requires the forwarded AW payload to match the captured write-address transaction.
ast_allowed_aw_is_issued	Issues the downstream write address after an effective write Allow decision.
ast_one_m_aw	Allows at most one downstream AW handshake per write transaction.
ast_m_aw_drops_after_handshake	Deasserts downstream AWVALID after the write-address handshake.
ast_denied_write_isolated	Prevents downstream write activity for a denied write.
ast_m_w_after_aw	Allows downstream WVALID only after an authorized write address has been forwarded and write data is pending.
ast_m_w_payload	Requires forwarded WDATA and WLAST to match the buffered upstream write beat.
ast_new_clean_w_strb	Preserves WSTRB on newly presented write beats before late revocation.
ast_new_revoked_w_poisoned	Forces WSTRB to zero on newly presented write beats after late revocation.
ast_m_w_count_conserved	Prevents the number of forwarded write beats from exceeding the number accepted upstream.
ast_allowed_w_stage_occupancy	Keeps the write skid-buffer occupancy consistent with accepted and forwarded beat counts.
ast_w_before_aw_stage_occupancy	Accounts correctly for buffered W-before-AW data before the write address arrives.
ast_policy_wait_w_stage_occupancy	Accounts correctly for buffered write data while authorization is pending.
ast_no_w_stage_overwrite	Prevents acceptance of a new upstream write beat while the one-entry write buffer is occupied.
ast_m_w_framing	Requires forwarded WLAST to occur exactly on the expected final write beat.
ast_buffered_w_is_issued_after_aw	Forwards a buffered write beat after the downstream write address is accepted.
ast_post_aw_w_is_issued	Forwards an accepted post-AW write beat once the authorized write path is ready.
ast_no_s_w_after_wlast	Stops accepting upstream write data after WLAST has been accepted.
ast_m_bready_only_when_expected	Accepts a downstream B response only after the authorized write address and all expected data beats complete.
ast_one_m_b_capture	Captures at most one downstream B response per write transaction.

Table X (continued)

Assertion	Description
ast_final_m_w_enables_b_capture	Enables downstream B-response capture after the final forwarded write beat.
ast_captured_b_is_presented	Presents a captured downstream B response to the upstream manager.
ast_s_b_has_owner	Presents BVALID only while a write transaction owner exists.
ast_s_b_id	Returns the original write transaction ID on the upstream B channel.
ast_local_denied_b	Requires a locally generated denied-write response to be DECERR after all expected upstream write beats and with no downstream write activity.
ast_local_denied_b_generated	Generates the local B response once a denied write has drained through WLAST.
ast_denied_final_w_bounded_response	Produces a denied-write B response within the specified bound after the final write beat.
ast_denied_aw_after_wlast_bounded_response	Produces a denied-write B response within the specified bound when denial follows an already completed write-data stream.
ast_forwarded_b	Returns the captured downstream B response only after complete authorized write forwarding.
ast_revoked_write_returns_decerr	Converts a write response to DECERR when late revocation applies before upstream response presentation.
ast_m_ar_only_if_allowed	Allows a downstream AR only for a captured, legal, policy-authorized read that has not already been forwarded.
ast_m_ar_payload	Requires the forwarded AR payload to match the captured read-address transaction.
ast_allowed_ar_is_issued	Issues the downstream read address after an effective read Allow decision.
ast_one_m_ar	Allows at most one downstream AR handshake per read transaction.
ast_m_ar_drops_after_handshake	Deasserts downstream ARVALID after the read-address handshake.
ast_denied_read_isolated	Prevents downstream read activity for a denied read.
ast_m_rready_only_when_expected	Accepts downstream read data only for an active authorized read when the local read buffer is available.
ast_registered_r_no_simultaneous_transfer	Prevents a downstream read beat from being captured and forwarded upstream in the same cycle.
ast_captured_r_is_presented	Presents each captured downstream read beat to the upstream manager.
ast_r_fifo_conservation	Keeps downstream and upstream read-beat counts consistent with at most one buffered beat.
ast_r_fifo_occupancy	Requires read-buffer validity to match the difference between captured and delivered read-beat counts.
ast_s_r_has_owner	Presents RVALID only while a read transaction owner exists.
ast_local_denied_r	Requires locally generated denied-read beats to return the original RID, zero data, DECERR, and correct RLAST with no downstream read activity.
ast_local_denied_r_generated	Continues generating local read beats while a denied read remains incomplete.
ast_denied_ar_bounded_response	Produces the first denied-read response beat within the specified bound after policy denial.
ast_forwarded_r_payload	Requires each allowed upstream read response to match the buffered downstream response payload.
ast_s_r_framing	Requires upstream RLAST to occur exactly on the expected final read beat.
ast_read_final_conservation	Requires an allowed read to complete with matching expected downstream and upstream beat counts.
ast_denied_read_final_conservation	Requires a denied read to complete with exactly the expected local beat count and no downstream read transfer.
ast_revoked_read_capture_is_poisoned	Converts a captured read beat to zero data and DECERR when late revocation applies.
ast_upstream_w_count_bounded	Prevents accepted upstream write beats from exceeding the AWLEN-derived expected count.
ast_firewall_w_count_bounded	Prevents firewall-to-subordinate write beats from exceeding the routed write's expected count.
ast_firewall_wlast_framing	Requires firewall-side WLAST to occur exactly on the expected final routed write beat.
ast_physical_write_beat_conservation	Requires each routed write beat to handshake with exactly the selected physical subordinate.
ast_local_write_has_no_physical_beats	Prevents physical-subordinate write beats for an unmapped subordinate handled locally.
ast_write_completion_conservation	Requires write completion counts to agree across manager, firewall, and routed-subordinate views.
ast_upstream_read_count_bounded	Prevents delivered upstream read beats from exceeding the ARLEN-derived expected count.
ast_firewall_read_count_bounded	Prevents firewall-side read beats from exceeding the routed read's expected count.

Table X (continued)

Assertion	Description
ast_physical_read_beat_conservation	Requires each routed read beat to handshake with exactly the selected physical subordinate.
ast_local_read_has_no_physical_beats	Prevents physical-subordinate read beats for an unmapped subordinate handled locally.
ast_read_completion_conservation	Requires an upstream read completion to contain exactly the expected number of beats.
ast_routed_read_completion_conservation	Requires routed-read completion counts to agree across subordinate, firewall, and manager views.
ast_initial_denied_write_returns_owner_decerr	Returns DECERR with the original write ID to the owner for an initial write denial or timeout.
ast_initial_denied_read_returns_owner_decerr	Returns zero-data DECERR beats with the original read ID and correct framing for an initial read denial or timeout.
ast_unimplemented_write_returns_owner_decerr	Returns DECERR with the original write ID when the requested subordinate is unimplemented.
ast_unimplemented_read_returns_owner_decerr	Returns zero-data DECERR with the original read ID when the requested subordinate is unimplemented.
ast_timeout_write_drains_only_owner	After write timeout, accepts remaining write data only from the locked transaction owner.
ast_timeout_read_framing	Requires locally generated timeout-read responses to use the original RID, zero data, DECERR, and correct RLAST.
Family 8: Subordinate Routing and Isolation	
Assertion	**Description**
ast_subordinate_map_decode_onehot	Requires each encoded SUBORDINATE_ID to decode to at most one protected-subordinate port.
ast_at_most_one_subordinate_address	Allows at most one protected subordinate to receive an AW or AR address, and never both address types simultaneously.
ast_at_most_one_subordinate_data_or_response	Allows write data, write-response readiness, and read-response readiness to target at most one protected subordinate.
ast_upstream_response_channel_mutex	Prevents simultaneous write and read responses on the firewall and manager-facing interfaces.
ast_write_address_decode_exact	Requires the physical AW-valid vector to equal the exact decode of the requested SUBORDINATE_ID.
ast_read_address_decode_exact	Requires the physical AR-valid vector to equal the exact decode of the requested SUBORDINATE_ID.
ast_unimplemented_aw_selects_no_subordinate	Routes an unimplemented write address to the local target and selects no physical subordinate.
ast_unimplemented_ar_selects_no_subordinate	Routes an unimplemented read address to the local target and selects no physical subordinate.
ast_subordinate_route_mutex	Prevents simultaneous active write and read subordinate routes.
ast_write_route_matches_captured_subordinate	Requires the locked write route and implementation status to match the captured SUBORDINATE_ID.
ast_read_route_matches_captured_subordinate	Requires the locked read route and implementation status to match the captured SUBORDINATE_ID.
ast_write_route_locked_until_manager_b	Keeps the write-subordinate route and transaction ID stable until the upstream B handshake completes.
ast_read_route_locked_until_manager_final_r	Keeps the read-subordinate route and transaction ID stable until the final upstream R handshake completes.
ast_write_data_route_exact	Routes WVALID only to the locked implemented write subordinate.
ast_write_response_ready_route_exact	Routes BREADY only to the locked implemented write subordinate.
ast_read_response_ready_route_exact	Routes RREADY only to the locked implemented read subordinate.
ast_no_write_route_is_quiet	Keeps write-data and write-response routing quiet when no write route is active.
ast_no_read_route_is_quiet	Keeps read-response routing quiet when no read route is active.
ast_aw_payload_to_selected_subordinate	Requires the selected protected subordinate to receive the exact firewall AW payload.
ast_aw_ready_from_selected_subordinate	Returns AWREADY from exactly the protected subordinate selected by SUBORDINATE_ID.
ast_w_payload_to_locked_subordinate	Requires the locked write subordinate to receive the exact firewall W payload.
ast_w_ready_from_locked_subordinate	Returns WREADY from the locked implemented write subordinate.
ast_b_from_locked_subordinate	Returns BVALID, BID, and BRESP from the locked implemented write subordinate.
ast_ar_payload_to_selected_subordinate	Requires the selected protected subordinate to receive the exact firewall AR payload.

Table X (continued)

Assertion	Description
ast_ar_ready_from_selected_subordinate	Returns ARREADY from exactly the protected subordinate selected by SUBORDINATE_ID.
ast_r_from_locked_subordinate	Returns RVALID and the complete R payload from the locked implemented read subordinate.
ast_unselected_subordinate_cannot_handshake_b	Prevents an unselected or unmapped subordinate from completing a write response handshake.
ast_unselected_subordinate_cannot_handshake_r	Prevents an unselected or unmapped subordinate from completing a read response handshake.
ast_no_physical_subordinate_before_effective_allow	Permits physical-subordinate AXI activity only after effective authorization of a legal, non-timed-out transaction.
ast_initial_denial_isolates_all_subordinates	Keeps every physical protected subordinate inactive after initial denial or timeout.
ast_illegal_allow_never_reaches_subordinate	Keeps every physical protected subordinate inactive when an externally allowed transaction is locally illegal.
ast_response_coincident_revoke_never_reaches_subordinate	Keeps every physical protected subordinate inactive when revocation coincides with an Allow response.
ast_timeout_never_reaches_subordinate	Keeps every physical protected subordinate inactive after policy timeout.
ast_unimplemented_write_uses_local_target	Ensures an unimplemented write uses only the local error target and no physical subordinate.
ast_unimplemented_read_uses_local_target	Ensures an unimplemented read uses only the local error target and no physical subordinate.
ast_unimplemented_write_local_b	Requires the local target to return DECERR with the routed write ID after all expected write beats.
ast_unimplemented_read_local_r	Requires the local target to return zero-data DECERR read beats with the routed ID and correct RLAST.
ast_unimplemented_read_exact_count	Requires an unimplemented read to complete with exactly the ARLEN-derived number of local response beats.
ast_late_write_revoke_keeps_subordinate	Keeps a late-revoked write bound to the same physical subordinate until completion.
ast_late_read_revoke_keeps_subordinate	Keeps a late-revoked read bound to the same physical subordinate until completion.

APPENDIX B
FORMAL ASSUMPTION CATALOG

The final integrated proof uses 60 elaborated SVA assumption instances derived from 12 assumption templates. These assumptions constrain only the external AXI4 managers, protected-side subordinates, and external policy interface. Six manager-side templates are instantiated for each of the six managers, producing 36 assumption instances, while three protected-side subordinate templates are instantiated for each of the seven protected-subordinate interfaces, producing 21 instances. The remaining three policy-interface assumptions are singleton properties. Table XI groups the assumption templates by constrained external interface and describes the environment behavior constrained by each one. In addition to the 60 elaborated SVA assumption instances, the formal setup contributes one saved-reset constraint, `rst_n == 1`, representing the post-reset proof state. VC Formal therefore reports a total of 61 formal constraints for the final integrated verification run.

Table XI
FORMAL ASSUMPTION CATALOG

Assumption	Description
Manager-Side AXI4 Contracts	
`asm_manager_aw_stable`	Holds `AWVALID` and the write-address payload stable while the manager is stalled by backpressure.
`asm_manager_w_stable`	Holds `WVALID`, `WDATA`, `WSTRB`, and `WLAST` stable while the write-data channel is stalled.
`asm_manager_ar_stable`	Holds `ARVALID` and the read-address payload stable while the manager is stalled by backpressure.
`asm_manager_wlast_after_aw`	Requires `WLAST` to identify exactly the expected final write-data beat after the corresponding write address has been accepted.
`asm_manager_wlast_with_aw`	Requires correct `WLAST` framing when the write address and a write-data beat are accepted in the same cycle.
`asm_manager_w_before_aw_framing`	Constrains write-data accepted before the write address so that the buffered beat count and observed `WLAST` remain consistent with the subsequently accepted `AWLEN`.
Protected-Side Subordinate Contracts	
`asm_subordinate_b_causal`	Allows a write response only for an active write after the expected number of write-data beats has been accepted, with `BID` matching the accepted `AWID`.
`asm_subordinate_r_stable`	Holds `RVALID` and the read-response payload stable while the read response is stalled by backpressure.
`asm_subordinate_r_causal`	Allows read data only for an active read, requires `RID` to match the accepted `ARID`, and requires `RLAST` only on the expected final beat.
Policy-Interface Contracts	
`asm_policy_rsp_causal`	Allows a policy response only for the active request after that request has been accepted or on the same cycle in which the request is accepted, and prevents responses after safe retirement.
`asm_policy_rsp_stable`	Holds a presented policy response and its `Allow` decision stable while the response is backpressured, unless cancellation retires the request.
`asm_policy_cancel_retires_stale_response`	Prevents a stale policy response from remaining valid after the timed-out policy generation has been permanently retired.

APPENDIX C
FORMAL COVER PROPERTY CATALOG

The cover properties complement the safety assertions by demonstrating reachability of representative normal, corner-case, denial, revocation, timeout, cancellation, arbitration, and routing scenarios under the stated environment assumptions. The final integrated configuration contains 103 active source-level cover templates that elaborate to 149 cover-property instances. Three active templates in the fixed-priority arbiter checker elaborate to 12 instances, 63 active firewall templates elaborate to 63 instances, and 37 integration-level templates elaborate to 74 instances because selected properties are instantiated per manager or per protected subordinate. Table XII catalogs the active cover templates and describes their reachability scenarios.

Table XII
FORMAL COVER PROPERTY CATALOG

Fixed-Priority Arbiter Reachability	
Cover Property	**Reachability Scenario**
`cov_grant_issued`	Shows that each requester can receive a grant.
`cov_adjacent_contention`	Exercises simultaneous requests from each adjacent priority pair.
`cov_all_reqs_active`	Exercises the extreme-contention case in which all requesters are active.
Firewall Write/Read Transaction Reachability	
Cover Property	**Reachability Scenario**
`cvr_m_b_capture`	Exercises capture of a protected-side write response by the firewall.
`cvr_clean_single_write`	Exercises a legal single-beat write that is allowed and completes without DECERR.
`cvr_allowed_w_before_aw`	Exercises an allowed write in which W is accepted before AW and later forwarded.
`cvr_denied_w_before_aw`	Exercises a W-before-AW write that is denied and completed locally with DECERR.
`cvr_simultaneous_aw_wlast_denied`	Exercises simultaneous AW and final W acceptance followed by policy denial and local DECERR.
`cvr_multibeat_write`	Exercises a legal allowed multibeat write through completion.
`cvr_w_backpressure`	Exercises protected-side write-data backpressure followed by a successful W handshake.
`cvr_revoke_before_w`	Exercises late write revocation before any protected-side W beat, causing zero-strobe forwarding and DECERR completion.
`cvr_revoke_during_w_stall`	Exercises late write revocation while a nonzero-strobe W beat is stalled, followed by DECERR completion.
`cvr_revoke_at_b_capture`	Exercises revocation coincident with capture of the protected-side B response, followed by DECERR upstream.
`cvr_allowed_multibeat_read`	Exercises a legal allowed multibeat read through the final read-data beat.
`cvr_revoke_before_first_r`	Exercises late read revocation before the first protected-side R beat, causing zero-data DECERR upstream.
`cvr_revoke_during_r_stall`	Exercises late read revocation while a nonfinal upstream R beat is stalled, then transitions to revoked completion.
`cvr_revoke_at_r_capture`	Exercises revocation coincident with protected-side R capture, followed by zero-data DECERR upstream.
`cvr_denied_multibeat_read`	Exercises a denied multibeat read that completes locally with DECERR through RLAST.
`cvr_denied_256_beat_read`	Exercises local DECERR completion of a denied maximum-length 256-beat read.
Firewall Authorization and Policy-Interface Reachability	
Cover Property	**Reachability Scenario**
`cvr_illegal_write_fail_closed`	Exercises an illegal write for which policy returns Allow but the firewall still completes locally with DECERR.
`cvr_illegal_read_fail_closed`	Exercises an illegal read for which policy returns Allow but the firewall still completes locally with DECERR.
`cvr_policy_req_backpressure`	Exercises multiple cycles of policy-request backpressure before request acceptance.
`cvr_policy_rsp_backpressure`	Exercises a backpressured policy response followed by response acceptance.
`cvr_delayed_write_allow`	Exercises a delayed Allow decision for a write request.
`cvr_delayed_write_deny`	Exercises a delayed Deny decision for a write request.
`cvr_delayed_read_allow`	Exercises a delayed Allow decision for a read request.
`cvr_delayed_read_deny`	Exercises a delayed Deny decision for a read request.
`cvr_w_before_aw_policy_latency`	Exercises a W-before-AW transaction that remains buffered while the policy decision is pending.

Table XII (continued)

Cover Property	Reachability Scenario
cvr_simultaneous_aw_w_policy_latency	Exercises simultaneous AW/W acceptance with buffered write data while the policy decision is pending.
cvr_policy_allow_changes_while_invalid	Shows that the policy Allow value may change while policy response VALID is low.
cvr_distinct_manager_ids	Exercises consecutive policy requests carrying different trusted MANAGER_ID values.
cvr_distinct_subordinate_ids	Exercises consecutive policy requests carrying different SUBORDINATE_ID values.
cvr_one_manager_different_subordinates	Exercises one manager issuing transactions to different protected subordinates.
cvr_one_subordinate_read_and_write	Exercises both read and write transactions to the same protected subordinate.
Firewall Revocation and Denial Reachability	
Cover Property	**Reachability Scenario**
cvr_valid_late_revocation	Exercises a late revocation after an initial write or read transaction has been allowed.
cvr_simultaneous_write_allow_revoke_local_deny	Exercises write Allow and revocation on the same policy-response handshake, producing local denial.
cvr_simultaneous_read_allow_revoke_local_deny	Exercises read Allow and revocation on the same policy-response handshake, producing local denial.
cvr_w_before_aw_simultaneous_revoke_local_deny	Exercises W-before-AW followed by simultaneous Allow/revocation and local write denial.
cvr_buffered_wlast_simultaneous_revoke_local_deny	Exercises simultaneous Allow/revocation when a buffered final write beat is present, followed by local DECERR.
cvr_simultaneous_aw_w_initial_revoke_local_deny	Exercises simultaneous AW/W acceptance followed by simultaneous Allow/revocation and local write denial.
cvr_predecision_revocation_is_ignored	Exercises a revocation before policy resolution and shows that no protected-side request is released.
cvr_revoke_during_b_stall	Exercises revocation while an already presented non-DECERR B response is stalled, preserving that presented response.
cvr_illegal_write_deny	Exercises policy Deny for an illegal write followed by local DECERR completion.
cvr_illegal_read_deny	Exercises policy Deny for an illegal read followed by local DECERR completion.
Firewall Timeout, Cancellation, and Quarantine Reachability	
Cover Property	**Reachability Scenario**
cvr_timeout_waiting_policy_req_ready	Exercises timeout while the policy request is still waiting for request acceptance, requiring no cancellation.
cvr_timeout_waiting_policy_rsp_valid	Exercises timeout after request acceptance while waiting for a policy response, entering cancellation quarantine.
cvr_timely_allow_before_timeout	Exercises an effective Allow response before the final permitted policy cycle.
cvr_timely_deny_before_timeout	Exercises a Deny response before the final permitted policy cycle.
cvr_allow_on_final_permitted_cycle	Exercises an Allow response on the final permitted cycle and confirms that the response wins over timeout.
cvr_deny_on_final_permitted_cycle	Exercises a Deny response on the final permitted cycle and confirms that the response wins over timeout.
cvr_write_timeout_without_buffered_w	Exercises write timeout with no buffered W beat, draining to WLAST and returning DECERR.
cvr_write_timeout_with_buffered_nonfinal_w	Exercises write timeout with a buffered nonfinal W beat, discarding it and completing locally with DECERR.
cvr_write_timeout_with_buffered_wlast	Exercises write timeout with a buffered final W beat and local DECERR completion.
cvr_w_before_aw_policy_timeout	Exercises W-before-AW followed by policy timeout and local write denial.
cvr_simultaneous_aw_w_policy_timeout	Exercises simultaneous AW/W acceptance followed by policy timeout and local write denial.
cvr_single_beat_read_timeout	Exercises timeout of a single-beat read and zero-data DECERR completion.
cvr_multibeat_read_timeout	Exercises timeout of a multibeat read and local DECERR completion through RLAST.
cvr_maximum_read_timeout_complete	Exercises timeout and complete local DECERR response for a 256-beat read.
cvr_late_response_safely_quarantined	Exercises a late policy response after timeout while response acceptance is blocked by quarantine.
cvr_policy_cancel_backpressure_then_retire	Exercises cancellation backpressure followed by a successful cancellation handshake.
cvr_retirement_fence_multiple_cycles	Exercises persistence of the retired-generation fence across multiple cycles.

Table XII (continued)

Cover Property	Reachability Scenario
cvr_new_transaction_after_safe_retirement	Exercises admission of a new transaction only after timed-out completion and cancellation retirement.
cvr_cancel_before_local_completion	Exercises cancellation retirement before the local AXI denial has completed.
cvr_cancel_with_local_completion	Exercises cancellation and local AXI completion occurring in the same cycle.
cvr_request_accepts_on_timeout_boundary	Exercises policy-request acceptance on the timeout boundary, requiring cancellation quarantine.
cvr_revocation_before_timeout_is_ignored	Exercises predecision revocation followed later by timeout, showing that the early revocation does not become late revocation.
Integrated Manager/subordinate Path Reachability	
Cover Property	**Reachability Scenario**
cvr_manager_wins_arbitration	Shows that each manager can win arbitration and have a transaction transfer accepted.
cvr_manager_aw_before_w	Exercises each manager completing a write with AW accepted before W.
cvr_manager_w_before_aw	Exercises each manager completing a write with W accepted before AW.
cvr_manager_simultaneous_aw_w	Exercises each manager completing a write with AW and W accepted together.
cvr_manager_policy_deny_returns_here	Exercises policy denial and verifies that DECERR completion returns to the owning manager.
cvr_allowed_write_to_subordinate	Exercises an allowed write through each protected subordinate and back to a manager response.
cvr_allowed_read_from_subordinate	Exercises an allowed read through each protected subordinate and back through the final manager read beat.
Integrated Arbitration, Identity, and Ownership Reachability	
Cover Property	**Reachability Scenario**
cvr_all_managers_request_highest_wins	Exercises simultaneous requests from all managers with the highest-priority manager selected.
cvr_lower_priority_wins_when_higher_idle	Shows that the lowest-priority manager can win when all higher-priority managers are idle.
cvr_back_to_back_different_managers	Exercises transaction completion followed immediately by admission of a different manager.
cvr_adjacent_transactions_no_cross_route	Exercises consecutive transactions from different managers through completion without cross-routing.
cvr_raw_aw_ar_contention_write_selected	Exercises simultaneous raw write/read contention while the write side is selected.
cvr_trusted_id_overwrites_spoof_attempt	Exercises a spoofed manager-supplied identity and shows replacement by the trusted physical-port MANAGER_ID.
cvr_queued_next_aw_is_blocked	Exercises a second AW from the current owner being blocked while the first write address remains active.
cvr_queued_next_ar_is_blocked	Exercises a second AR from the current owner being blocked while the first read address remains active.
Integrated Denial and Revocation Reachability	
Cover Property	**Reachability Scenario**
cvr_policy_deny_write_owner_decerr	Exercises write policy denial and DECERR completion to the transaction owner.
cvr_policy_deny_read_owner_decerr	Exercises read policy denial and final DECERR completion to the owner.
cvr_illegal_write_owner_decerr	Exercises policy Allow of an illegal write while the architecture still returns DECERR to the owner.
cvr_illegal_read_owner_decerr	Exercises policy Allow of an illegal read while the architecture still returns DECERR to the owner.
cvr_response_revoke_write_owner_decerr	Exercises write Allow coincident with revocation and DECERR completion to the owner.
cvr_response_revoke_read_owner_decerr	Exercises read Allow coincident with revocation and DECERR completion to the owner.
cvr_late_write_revoke_same_owner_subordinate	Exercises late write revocation while preserving the locked owner/subordinate association through DECERR completion.
cvr_late_read_revoke_same_owner_subordinate	Exercises late read revocation while preserving the locked owner/subordinate association through final read completion.
Integrated Timeout and Cancellation Reachability	
Cover Property	**Reachability Scenario**
cvr_timeout_before_policy_request_acceptance	Exercises timeout before policy-request acceptance without entering cancellation quarantine.
cvr_timeout_after_policy_request_acceptance	Exercises timeout after policy-request acceptance and entry into cancellation quarantine.

Table XII (continued)

Cover Property	Reachability Scenario
cvr_policy_cancellation_backpressure	Exercises cancellation backpressure followed by cancellation retirement in the integrated architecture.
cvr_local_completion_before_cancellation	Exercises local AXI completion before cancellation retirement for an accepted timed-out policy request.
cvr_cancellation_before_local_completion	Exercises cancellation retirement before the local AXI completion of a timed-out transaction.
cvr_new_manager_after_cancellation_retirement	Exercises admission of a new manager after cancellation retirement releases the ownership fence.
cvr_contention_with_policy_timeout_quarantine	Exercises multi-manager contention followed by an accepted-request timeout and quarantine.
cvr_contention_with_policy_request_backpressure	Exercises multi-manager contention together with policy-request backpressure before acceptance.
Integrated Structural Routing and Stress Reachability	
Cover Property	**Reachability Scenario**
cvr_unimplemented_write_structural_error	Exercises an authorized write to an unmapped subordinate and local structural DECERR propagation to the manager.
cvr_unimplemented_read_structural_error	Exercises an authorized read to an unmapped subordinate and local structural DECERR propagation through RLAST.
cvr_one_manager_different_subordinates	Exercises the same physical manager accessing different protected subordinates across transactions.
cvr_different_managers_same_subordinate	Exercises different physical managers accessing the same protected subordinate across transactions.
cvr_maximum_length_write_one_owner_subordinate	Exercises a 256-beat write while preserving one owner/subordinate route through completion.
cvr_maximum_length_read_one_owner_subordinate	Exercises a 256-beat read while preserving one owner/subordinate route through the final beat.

The catalog lists only cover templates active in the reported integrated configuration with six managers, seven protected subordinates, a 32-cycle policy timeout, and integration mode enabled. Five additional source-level cover statements are inactive in this configuration and are therefore excluded: cov_yield_to_lower and cvr_aw_ar_contention, which are standalone-mode covers; cvr_one_cycle_simultaneous_aw_wlast_timeout, which applies only to a one-cycle timeout; and cvr_two_cycle_stalled_request_timeout and cvr_two_cycle_boundary_request_quarantine, which apply only to a two-cycle timeout. The 103 active templates elaborate to the 149 cover properties reported by VC Formal.